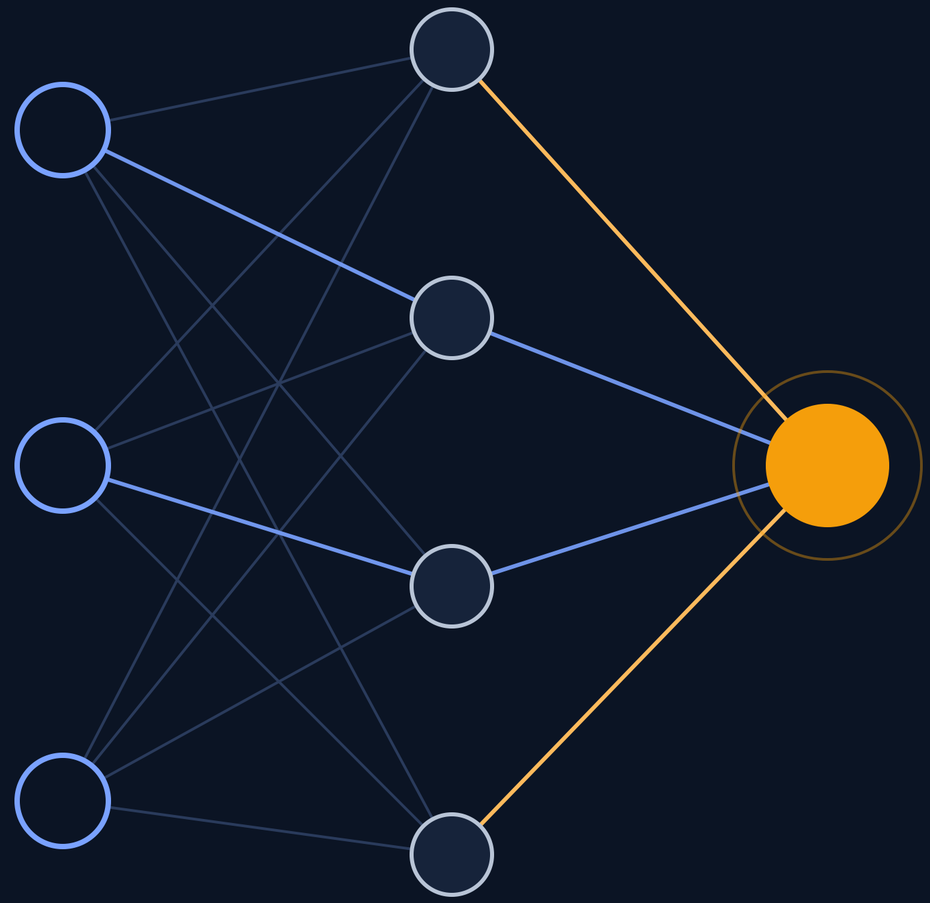
CLASE 1 NEURAL NETWORKS: DEEP LEARNING · UIDE

Fundamentos de redes neuronales

Qué es la inteligencia artificial, cómo aprende una neurona y cómo entrenar tu primera red



Andrés Ordóñez-Bolaños, MSc

Maestría en Ciencia de Datos y Máquinas de Aprendizaje · Viernes 23 y sábado 24 de octubre de 2026

Antes de empezar: quiénes estamos aquí

SU DOCENTE

Andrés Ordóñez-Bolaños

Lead XR Software Engineer

MSc en Ciencias de la Computación

+7 años en videojuegos, realidad extendida y gemelos digitales

USTEDES · RESPONDAN EN EL CHAT

1. ¿En qué sector trabajan?
2. Python, del 1 al 5: ¿qué tan cómodos se sienten?
3. ¿Han usado alguna vez inteligencia artificial en su trabajo?
4. Un problema de su trabajo o de Ecuador que les gustaría resolver con datos

GUÍA 00 · Encuadre

Cómo leer estas láminas

CONCEPTO
La idea central que deben llevarse

ANALOGÍA
La misma idea con un ejemplo de la vida real

PRÁCTICA
Qué notebook abrir en Colab y qué deberían ver

PARA CURIOSOS
Más matemática, opcional: no entra al test

✓ Regla de la clase: **primero la intuición, después la fórmula**. Si una ecuación no se entiende, pregunten.

Cuatro semanas, un proyecto

SEMANA 1 · HOY
Fundamentos y Keras
19–25 oct

SEMANA 2
Arquitecturas: DNN, CNN, RNN
26 oct–1 nov

SEMANA 3
Modelos robustos y explicables
2–8 nov

SEMANA 4
Cierre y entrega final
9–12 nov · sin clase en vivo

CADA SEMANA VALE 100 PUNTOS

70	20	10
mini-proyecto en grupo	test individual	lectura y encuesta

APRENDIZAJE BASADO EN PROYECTOS

Cada semana construye un piso del **proyecto de titulación**. Semanas 1–3: 80 % de la nota · entrega final: 20 %.

Esta semana, paso a paso

AL TERMINAR LA SEMANA PODRÁN…

- Explicar qué es la IA, el machine learning y el deep learning
- Calcular a mano lo que hace una neurona
- Explicar cómo aprende una red: pérdida, gradiente y retropropagación
- Entrenar su primera red en Keras con datos reales de su proyecto

Lun 19	Lectura y encuesta en Canvas
Mié 21	Abre la Clase 1
Vie 23	Sesión 1 · de la IA a la neurona
Sáb 24	Sesión 2 · redes, Keras y proyecto
Mar 27	Cierre: test, encuesta y entregable

PRE-QUIZ 00 · Encuadre

Pre-quiz: ¿de dónde partimos?

ANTES DE EMPEZAR

¿Qué sabemos ya? Un juego de 5 minutos

Diez preguntas rápidas, 20 segundos cada una, al estilo Kahoot: cinco de bases y cinco **ideas clave** de la semana.

No es nota. Es el punto de partida: el sábado las cinco ideas clave vuelven en el quiz final y vemos cuánto avanzamos.

10
preguntas

20 s
por pregunta

0
puntos en la nota

Escanea y juega

abrir el pre-quiz ↗

01

¿De qué hablamos cuando hablamos de IA?

Inteligencia artificial, machine learning y deep learning, desde cero y con ejemplos

Viernes **01** 02 03 04 Sábado 05 06 07 08 09

La IA ya está en tu día

Mapas y tráfico
Estima cuánto tardarás según el tráfico de este momento

Música y series
Te recomienda qué ver o escuchar después

Tu banco
Bloquea una compra rara con tu tarjeta antes de que sea fraude

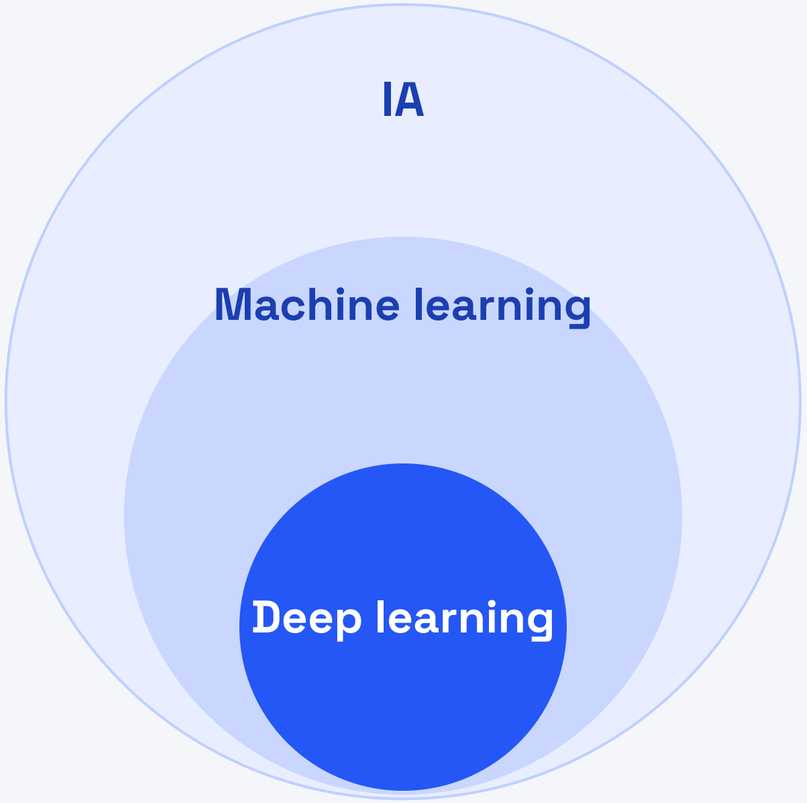
Tu correo
Separa el spam sin que nadie escriba una regla por cada mensaje

Tu celular
Se desbloquea al reconocer tu cara

Asistentes de IA
Traducen, resumen y responden preguntas

Lo que tienen en común: **aprendieron patrones a partir de datos**, no de reglas escritas a mano.

IA, machine learning y deep learning



INTELIGENCIA ARTIFICIAL

Máquinas que hacen tareas que asociamos con inteligencia, con reglas o aprendiendo.

Ej.: un programa que juega ajedrez

MACHINE LEARNING

La máquina **aprende patrones de los datos** en vez de seguir reglas escritas.

Ej.: el filtro de spam de tu correo

DEEP LEARNING · ESTE CURSO

Machine learning con **redes neuronales de muchas capas** que descubren sus propias características.

Ej.: reconocer tu cara en el celular

Programar reglas o aprender de ejemplos

PROGRAMACIÓN TRADICIONAL



MACHINE LEARNING



Spam, a la antigua

“Si el asunto dice GRATIS, es spam.” Los estafadores escriben GR4TIS y la regla falla.

Spam, con machine learning

Le mostramos miles de correos marcados “spam / no spam” y descubre los patrones solo.

Aprender de ejemplos: así se ven los datos

CARACTERÍSTICAS (x) · lo que sabemos del socio

ETIQUETA (y) · lo que queremos predecir

Provincia	Sector	Ingreso (USD)	Meses de socio	Atrasos	Garante	¿Cayó en mora?
Pichincha	agricultura	303	10	0	no	no (0)
Chimborazo	comercio	516	11	0	no	no (0)
Azuay	comercio	623	43	0	sí	no (0)
Chimborazo	comercio	618	8	0	no	**sí (1)**
Chimborazo	artesanía	348	13	0	no	**sí (1)**

Cada fila es un ejemplo. La cooperativa ficticia “Valle Andino” tiene 8 000.

Aprender = encontrar la relación entre x y y para predecir socios nuevos.

CONCEPTO 01 · ¿De qué hablamos?

Tres formas de aprender

ESTE CURSO

Supervisado

Aprende de ejemplos **con la respuesta correcta**.

Como estudiar con el solucionario al lado.

Ej.: ¿este socio caerá en mora? ¿este correo es spam?

No supervisado

Busca grupos y patrones **sin respuestas**.

Como ordenar un clóset sin que nadie te diga las categorías.

Ej.: agrupar clientes con hábitos parecidos

Por refuerzo

Aprende probando, con **premios y castigos**.

Como entrenar a un perro con premios.

Ej.: robots, juegos, sistemas de control

La idea es de 1958. ¿Por qué despegó ahora?

Datos

Internet, celulares y sensores generan millones de ejemplos.

\+

Cómputo

Las GPU, nacidas para videojuegos, hacen miles de cuentas a la vez.

\+

Algoritmos

Retropropagación, ReLU y buenas inicializaciones: lo que veremos esta semana.

2012 Los tres ingredientes coinciden: una red profunda (AlexNet) gana por amplio margen el gran concurso de reconocimiento de imágenes, ImageNet.

¿Cuándo conviene el deep learning?

Tipo de dato	Ejemplo	¿Deep learning?
Imágenes	Fotos de cultivos con plagas, radiografías	**Sí, casi siempre**
Texto	Reseñas y chats de clientes	**Sí**
Audio y voz	Llamadas a un call center	**Sí**
Series largas	Consumo eléctrico por hora	**Muchas veces**
Tablas	8 000 créditos con 13 columnas	**No siempre: compara con un modelo clásico**

Regla práctica: el deep learning brilla cuando es difícil describir a mano qué mirar en los datos. Siempre compárenlo con una línea base sencilla.

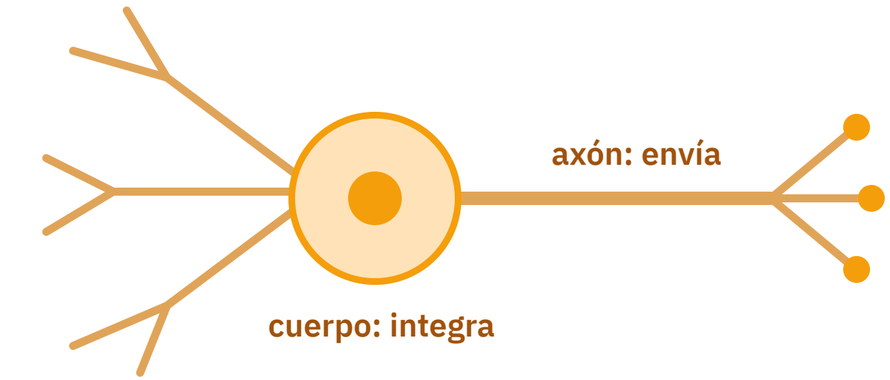
02

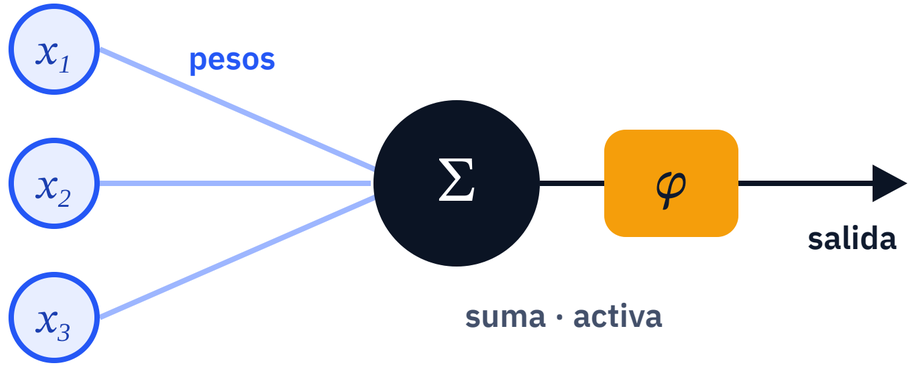
La neurona artificial

La pieza básica de toda red: multiplica, suma y decide

Viernes 01 **02** 03 04 Sábado 05 06 07 08 09

De la neurona biológica a la artificial





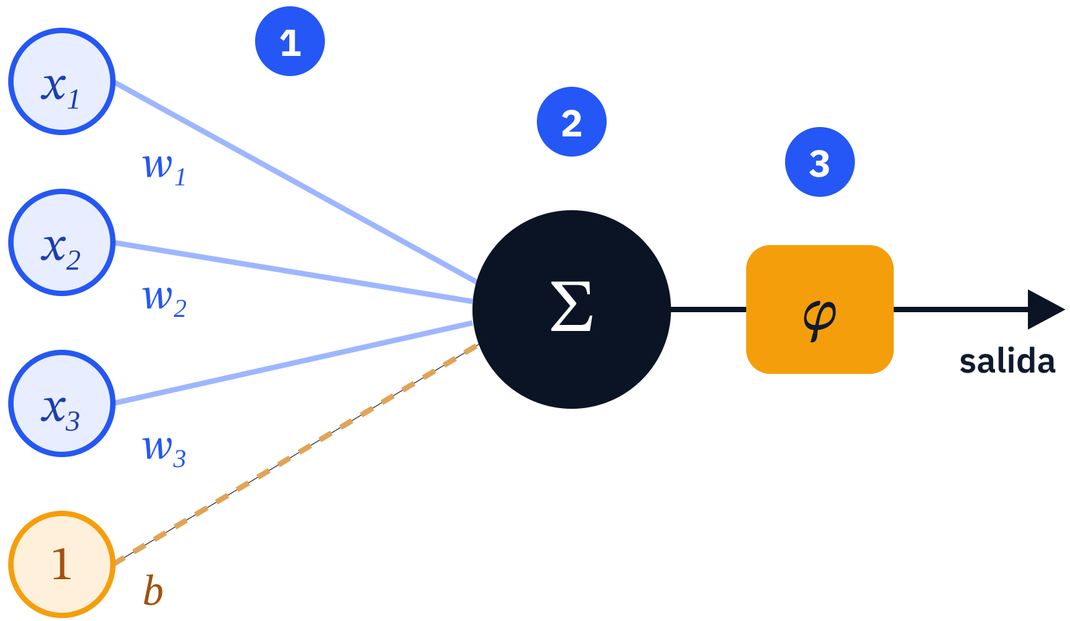
Dendritas
→ **entradas x**

Fuerza de la conexión
→ **pesos w**

El cuerpo integra
→ **suma Σ**

Se “dispara” o no
→ **activación φ**

Una neurona hace tres cosas



1 Multiplica
cada dato por su peso: cuánto importa

2 Suma
todo, más un sesgo *b* que sube o baja la exigencia

3 Decide
con una función de activación φ qué sale

La ecuación, término por término

$$z = w_1x_1 + w_2x_2 + w_3x_3 + b$$

- z: puntaje total
- w_1x_1: dato 1 × su importancia
- w_2x_2: dato 2 × su importancia
- w_3x_3: dato 3 × su importancia
- b: sesgo: la exigencia

$$a = \varphi(z)$$

La **activación** φ convierte el puntaje en la salida de la neurona.

FORMA CORTA Y EN PYTHON

$z = \mathbf{w} \cdot \mathbf{x} + b \longrightarrow$ `z = w @ x + b`

Una neurona es un comité de crédito

Mira la ficha

Ingreso, antigüedad y atrasos del socio

entradas x

Pondera

Confía más en unas señales que en otras; los atrasos restan

pesos w

Aplica su exigencia

Un comité conservador pide más antes de aprobar

sesgo b

Decide

Aprueba o no; o da una probabilidad de pago

activación φ

Aprender = ajustar cuánto confía el comité en cada dato, después de ver quién pagó y quién no.

Una neurona, paso a paso

Ingreso	0.8×1.2	+0.96
Antigüedad	0.5×0.7	+0.35
Atrasos	$(-1.0) \times (-2.0)$	+2.00
Exigencia	b	−0.50
Puntaje z	$0.96 + 0.35 + 2.00 - 0.50 =$	**2.81**

ESCALÓN

Aprueba

porque z es mayor que 0

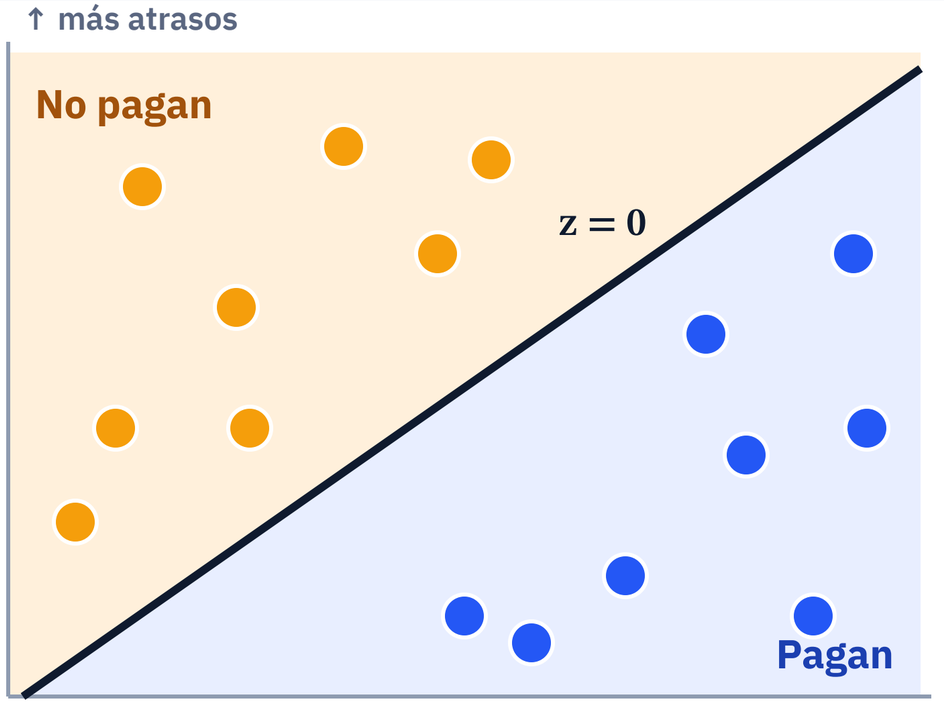
SIGMOIDE

94 %

probabilidad de que pague

Lo que más pesó: tener **menos atrasos** que el promedio (negativo × negativo = positivo).

Una neurona dibuja una línea



1 **Un lado aprueba**
Donde z es positivo: zona azul, “paga”

2 **El otro lado niega**
Donde z es negativo: zona naranja, “no paga”

3 **La línea es la frontera**
Exactamente donde z = 0: la neurona duda

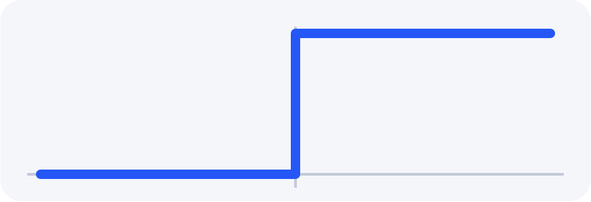
EN FÓRMULA

$$w_1 \cdot \text{ingreso} + w_2 \cdot \text{atrasos} + b = 0$$

Los **pesos** giran la línea; el **sesgo** la mueve. Entrenar = buscar la mejor posición.

La activación: cómo decide la neurona

Escalón

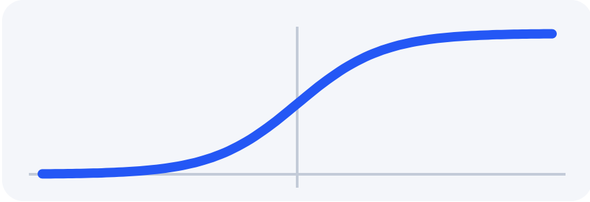


$1 \text{ si } z \geq 0; \text{ si no, } 0$

Un interruptor de luz: solo sí o no, 0 o 1.

El perceptrón clásico

Sigmoide

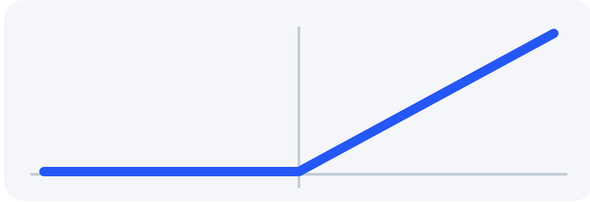


$\sigma(z) = 1 \div (1 + e^{-z})$

Un regulador de intensidad: un valor suave entre 0 y 1, una probabilidad.

La salida “¿paga o no?”

ReLU



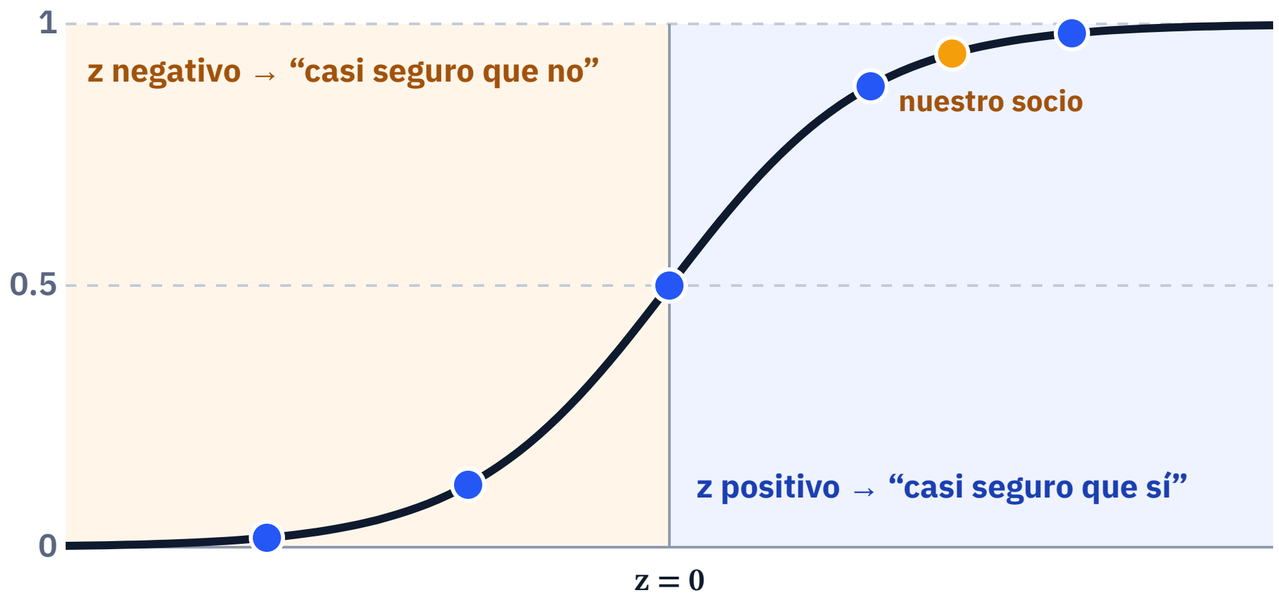
$\text{ReLU}(z) = \text{máx}(0, z)$

Una válvula de agua: si z es negativo da 0; si es positivo, pasa igual.

Capas ocultas: la más usada hoy

¿Para qué sirve? Sin activación, apilar neuronas sería solo sumar sumas: la red nunca aprendería curvas.

La sigmoide: de puntaje a probabilidad



PUNTAJE → PROBABILIDAD

$z = -4$	2 %
$z = -2$	12 %
$z = 0$	50 %
$z = 2$	88 %
$z = 2.81$	94 %
$z = 4$	98 %

LA FÓRMULA (NO HAY QUE MEMORIZARLA)

$$\sigma(z) = 1 \div (1 + e^{-z})$$

PAUSA ACTIVA 02 · Verificación rápida

¿Quedó clara la neurona?

1 Si el peso del **ingreso** fuera 0, ¿qué significaría?

2 El comité se vuelve **más exigente**. ¿Qué número de la neurona cambia?

3 Con sigmoide, un socio tiene **z = 0**. ¿Qué probabilidad le da?

4 ¿Qué activación usarías para dar una **probabilidad**: escalón, sigmoide o ReLU?

Respondan en el chat de Teams con el número de la pregunta. Tienen 3 minutos.

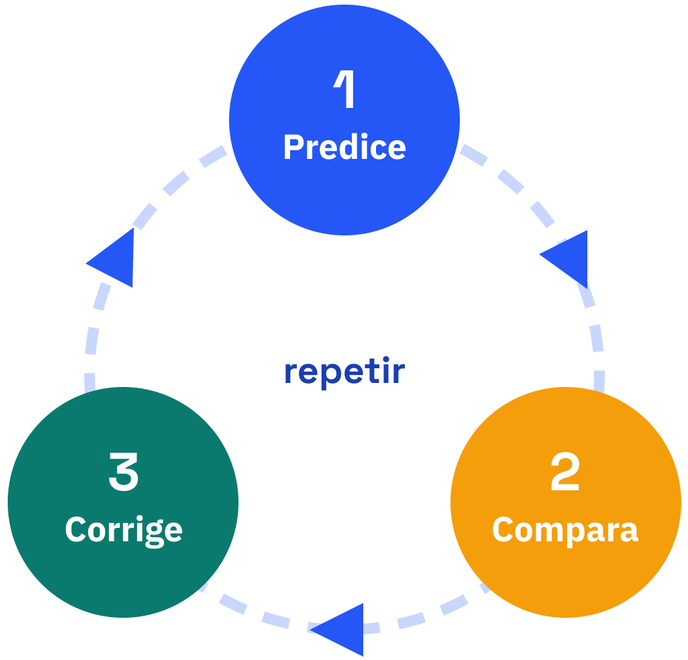
03

Cómo aprende una neurona

El perceptrón: equivocarse, corregir y repetir. Y su primer límite

Viernes 01 02 **03** 04 Sábado 05 06 07 08 09

Aprender = equivocarse y corregir



COMO AJUSTAR LA SAL DEL LOCRO

Pruebas una cucharada	→	**predecir**
¿Le falta o le sobra sal?	→	**comparar**
Echas una pizca (o agua)	→	**corregir**
Vuelves a probar	→	**otra época**

Cada vuelta corrige **un poquito**: nadie echa el salero entero.

La regla del perceptrón: corregir solo cuando falla

$$w_{nuevo} \leftarrow w + \eta \cdot y \cdot x$$

- w_{nuevo}: el peso corregido
- w: el peso actual
- η: tamaño del paso (aquí 1)
- y: respuesta correcta: +1 o −1
- x: el dato

Si acierta

No toca nada. Lo que funciona no se arregla.

Si se equivoca

Empuja los pesos **hacia la respuesta correcta**, en la dirección del dato.

EL SESGO, IGUAL

$$b \leftarrow b + \eta \cdot y$$

como si su dato valiera siempre 1

En el comité: si aprobó a alguien que no pagó, desconfía un poco más de los datos que lo hicieron aprobar.

Una corrección, con números

1 · ANTES

pesos w	$(0.2, -0.5)$
sesgo b	0
socio x	$(1, 1)$
respuesta real y	$+1$ (pagó)
$z = 0.2 - 0.5 + 0$	-0.30

Niega × se equivocó

2 · CORRIGE

con paso $\eta = 1$ y respuesta $y = +1$

$$w_1 = 0.2 + 1 \cdot (+1) \cdot 1 = 1.2$$

$$w_2 = -0.5 + 1 \cdot (+1) \cdot 1 = 0.5$$

$$b = 0 + 1 \cdot (+1) = 1$$

Como debía decir “sí”, le **sumamos** el dato a los pesos.

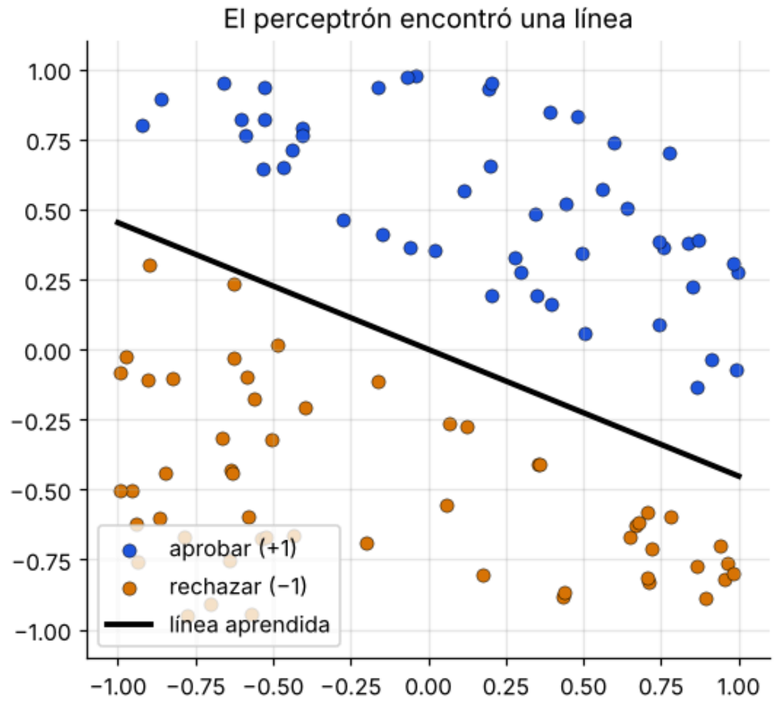
3 · DESPUÉS

pesos w	$(1.2, 0.5)$
sesgo b	1
mismo socio x	$(1, 1)$
respuesta real y	$+1$ (pagó)
$z = 1.2 + 0.5 + 1$	$+2.70$

Aprueba ✓ acertó

</> **Abre en Colab:** NB01 · sección 3 · El perceptrón aprende de UN error

Cuando se puede, el perceptrón lo logra



LA GARANTÍA

Si una línea puede separar los datos, el perceptrón la encuentra.

Lo hace en un número **finito** de correcciones. No adivina: está garantizado (Novikoff, 1962).

EN EL NB01 · SECCIÓN 4

2 → 0

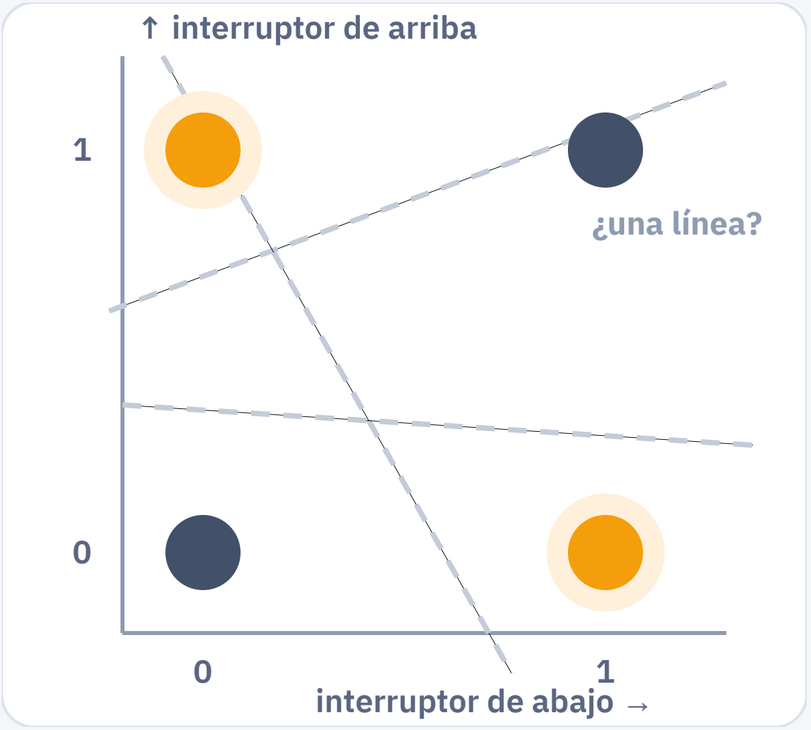
errores por época: en la segunda vuelta ya no falla

PARA CURIOSOS

$$\text{correcciones} \leq (R \,/\, \gamma)^2$$

γ es el **margen**: el ancho del “pasillo” entre clases. Pasillo ancho → aprende rápido.

El primer límite: XOR, la luz de la escalera



LA LUZ DE LA ESCALERA

Dos interruptores, uno abajo y otro arriba. La luz se enciende si están en posiciones **distintas**.

abajo	arriba	
0	0	Apagada
0	1	Encendida
1	0	Encendida
1	1	Apagada

Ninguna línea separa encendidas de apagadas. La solución llega mañana: **más de una neurona**.

COLAB 03 · Práctica

Todo el material, a un clic: abrir en Colab

CÓMO ABRIR UN NOTEBOOK

1. **Entra a la página de la semana**
   escanea el QR o abre el enlace
2. **Clic en «Abrir en Colab»**
   en el notebook del día; entra con tu cuenta de Google
3. **Archivo → Guardar una copia en Drive**
   si no, al cerrar la pestaña se pierden tus cambios
4. **Ejecuta con Shift + Enter**
   primero la celda Configuración; luego en orden

Todo el material

abrir la página ↗

NB01 viernes
Tu primera neurona
abrir NB01 ↗

NB02 sábado
Tu primera red
abrir NB02 ↗

NB03 mini-proyecto
La cooperativa
abrir NB03 ↗

PRÁCTICA 03 · Práctica 1 · 20 minutos

Manos a la obra: NB01, secciones 1 a 5

	EJEMPLO RESUELTO · YA ESTÁ HECHO	TU TAREA · ✅ O ❌
1	**Una neurona es una calculadora** deberías ver: z = 2.81	**T1 · Programa tu neurona**
2	**La activación: el interruptor** deberías ver: sigmoide 0.943	**T2 · De puntaje a decisión**
3	**El perceptrón aprende de un error** deberías ver: −0.30 → +2.70	**T3 · Corrige un error al revés**
4	**Entrenar un perceptrón completo** deberías ver: errores 2 → 0	**T4 · Aprende la compuerta AND**
5	**El límite: XOR** deberías ver: 4 errores siempre	**T5 · ¿Por qué falla?**

NB01 en Colab

abrir NB01 ↗

Ejecuta primero la celda **Configuración**.

En cada sección: **lee y ejecuta** el ejemplo, **predice** qué saldrá en tu tarea y luego comprueba con **Revisar**.

04

Medir el error y mejorar

La pérdida dice qué tan mal vamos; el gradiente, hacia dónde mejorar

CONCEPTO 04 · Medir el error

La pérdida: un solo número que mide el error

PRECIO DE CASAS EN QUITO · MILES DE USD

	real	predicho	error	error²
Casa 1	120	110	10	**100**
Casa 2	95	100	−5	**25**
Casa 3	150	170	−20	**400**
Casa 4	80	82	−2	**4**

Promedio de los errores²: **529 ÷ 4 = 132.25**

ANALOGÍA

La pérdida es la nota al revés

0 es perfecto. Mientras más alta, peor. Entrenar = bajarla.

SE LLAMA MSE

$$\text{MSE} = (1/N) \cdot \Sigma\,(y - \hat{y})^2$$

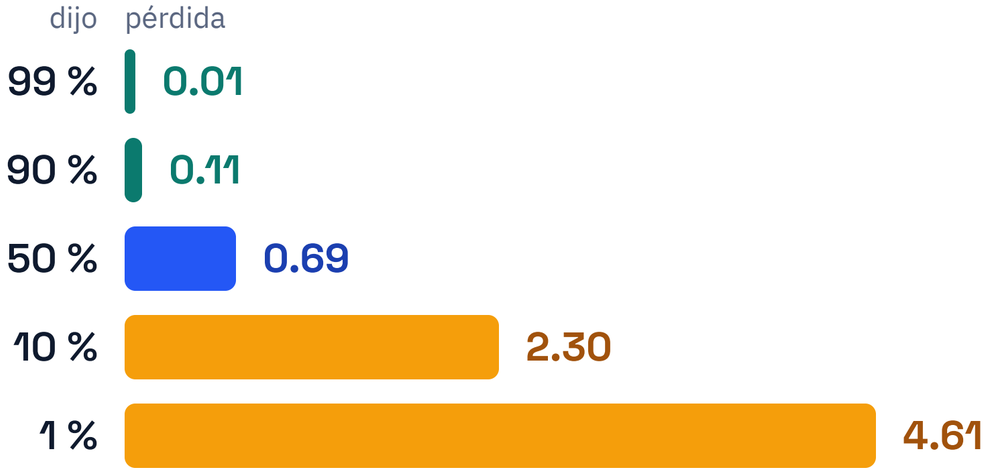
En palabras: el promedio de los errores al cuadrado.

Elevar al cuadrado castiga mucho los errores grandes: errar por 20 cuesta **400**.

Para clasificar: castigar la falsa seguridad

UN SOCIO QUE SÍ PAGÓ

¿Cuánto “cuesta” cada respuesta del modelo?



Equivocarse **con mucha seguridad** cuesta muchísimo.

ANALOGÍA

El pronóstico del clima en Quito

Si dices “99 % sin lluvia” y cae un aguacero, quedas muy mal. Si dijiste 50 %, no tanto.

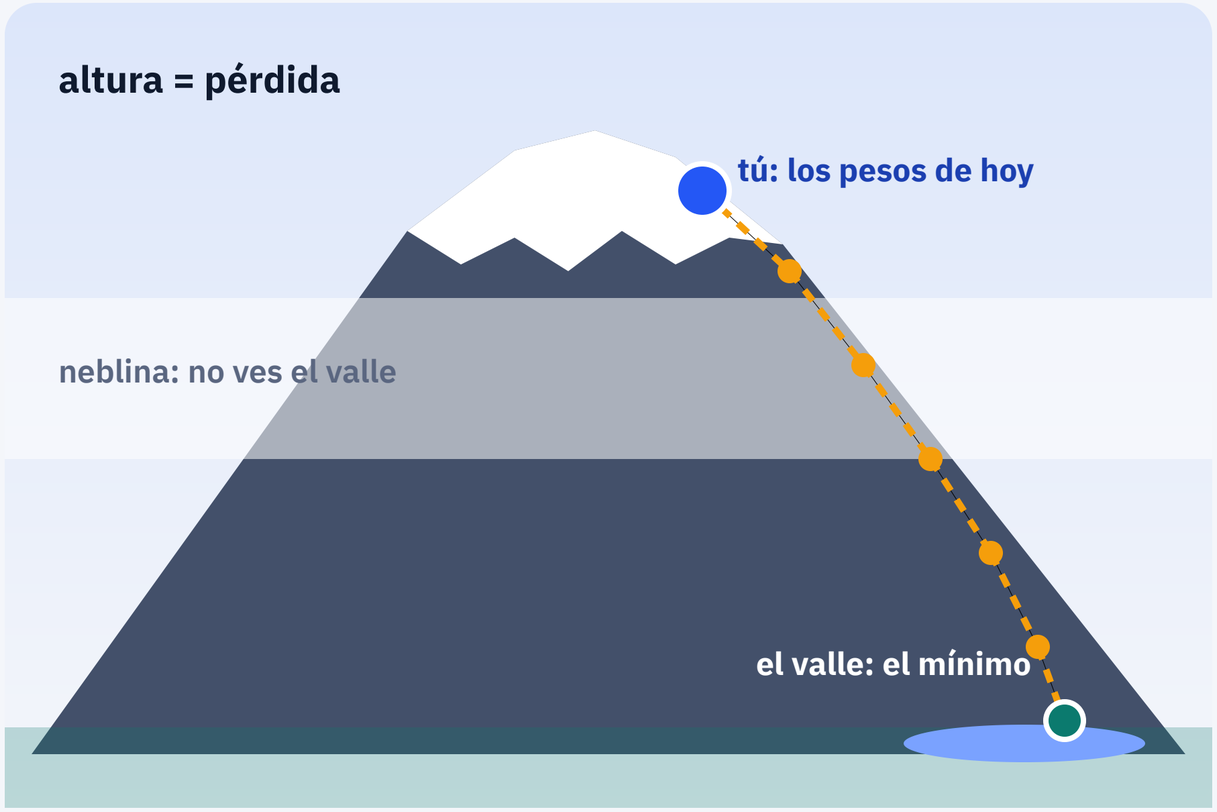
SE LLAMA ENTROPÍA CRUZADA (BCE)

pagó: pérdida $= -\log(\hat{y})$

no pagó: pérdida $= -\log(1-\hat{y})$

$\hat{y}$ = probabilidad que dio el modelo

Bajar el Cotopaxi con neblina



1 Tantea el suelo

con el pie: ¿hacia dónde baja más? Eso es la **pendiente** (el gradiente)

2 Da un paso

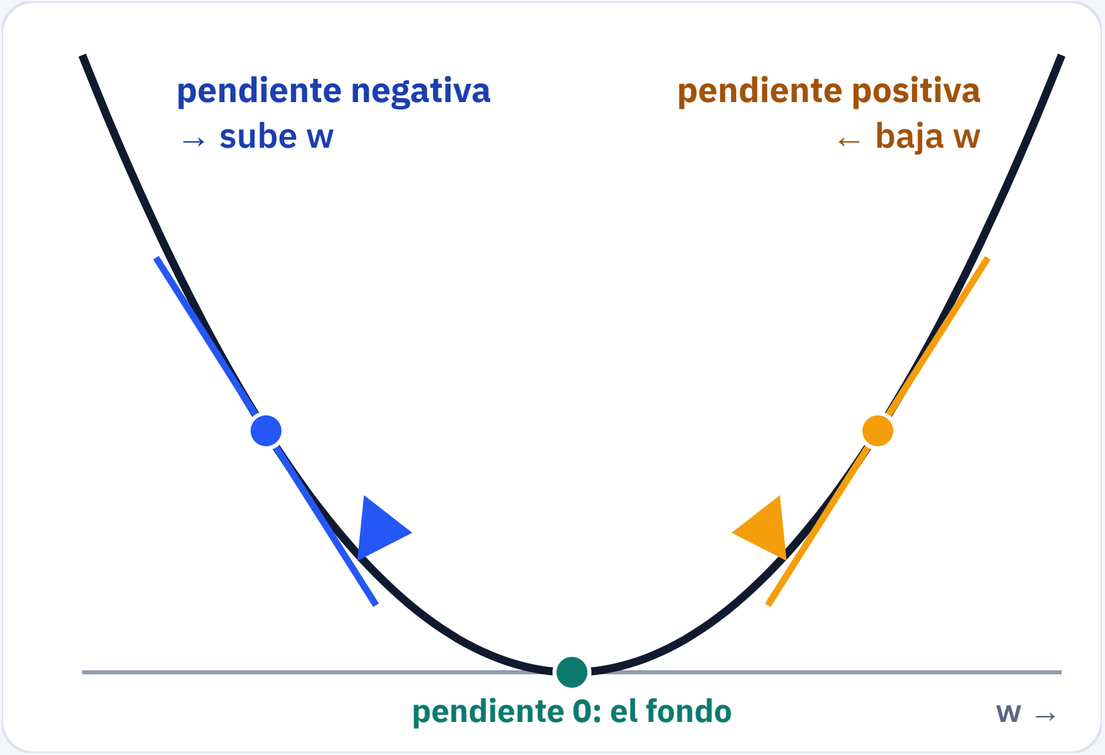
hacia abajo. El largo del paso es **η**, la tasa de aprendizaje

3 Repite

hasta que el suelo quede plano: llegaste a un **mínimo**

No necesitas ver el mapa completo: basta sentir la pendiente donde estás.

La pendiente dice hacia dónde bajar



$$w_{\text{peso nuevo}} \leftarrow w_{\text{peso actual}} - \eta_{\text{largo del paso}} \cdot \underbrace{\text{pendiente}}_{\text{hacia dónde sube}}$$

El **signo menos** es lo único que hay que recordar: la pendiente apunta cuesta arriba, nosotros vamos al revés.

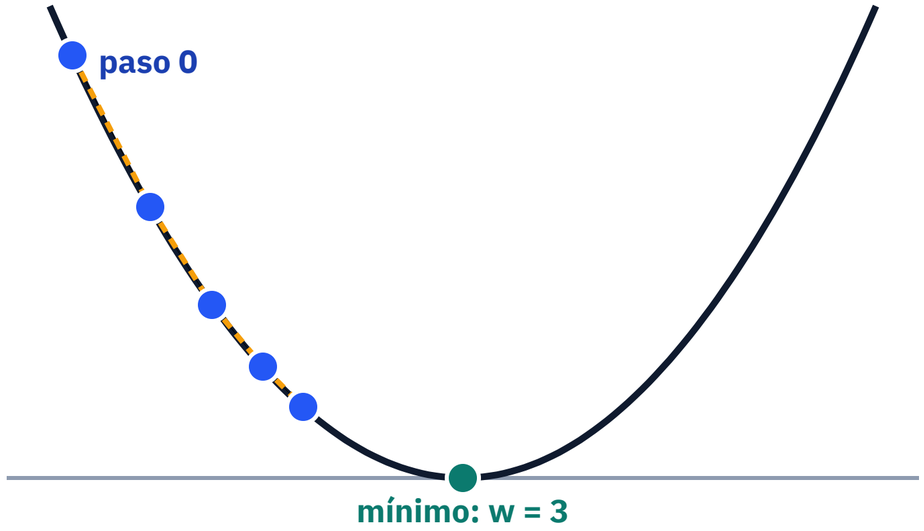
Bajando paso a paso, con números

CUATRO PASOS A MANO

pérdida $(w - 3)^2$ · paso $\eta = 0.1$ · arrancamos en $w = 0$

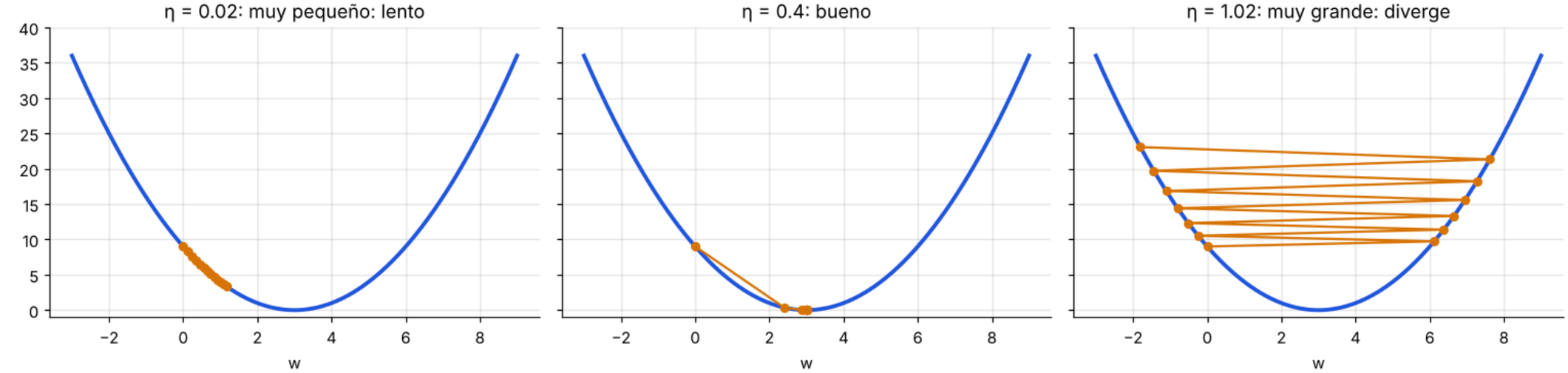
paso	w	pendiente	pérdida
0	**0**	−6	**9.00**
1	**0.6**	−4.8	**5.76**
2	**1.08**	−3.84	**3.69**
3	**1.464**	−3.07	**2.36**
4	**1.771**	−2.46	**1.51**

Los pasos se acortan solos: cerca del fondo la pendiente es más suave. Sigue así hasta $w \approx 3$.



</> **Abre en Colab:** NB01 · sección 7 · Descenso de gradiente en una dimensión

El largo del paso η: ni muy corto ni muy largo



η = 0.02 Muy corto

Llega, pero tarda una eternidad. Pasos de tortuga.

η = 0.4 Bien elegido

Llega al fondo en pocos pasos. Es lo que buscamos.

η = 1.02 Muy largo

Salta de lado a lado y cada vez queda más lejos: **diverge**.

PRÁCTICA 04 · Práctica 2 · 10 minutos en clase

Manos a la obra: NB01, secciones 6 a 8 y Mini-reto 1

	EJEMPLO RESUELTO · YA ESTÁ HECHO	TU TAREA · ✅ O ❌
6	**La pérdida: MSE y BCE** deberías ver: MSE 132.25 · BCE 0.01 → 4.61	**T6 · Programa MSE y BCE**
7	**Descenso de gradiente** deberías ver: w: 0 → 0.6 → … → 3	**T7 · Baja otra montaña**
8	**El tamaño del paso η** deberías ver: 0.02 lento · 0.4 bien · 1.02 diverge	**T8 · ¿Cuáles convergen?**

MINI-RETO 1 · TU PROPIO COMITÉ DE CRÉDITO

Seis socios, dos datos (ingreso y atrasos). Elige a mano los pesos y el sesgo para que tu neurona apruebe a los que pagaron y rechace a los que no. La celda dibuja tu frontera y cuenta aciertos: **¿logras 6 de 6?**

NB01 en Colab

abrir NB01 ↗

Lo que no alcances, termínalo en casa.

CIERRE Viernes · lo que nos llevamos

La receta de todo el deep learning

1

Predecir

la neurona calcula z y la activación

$$\hat{y} = \sigma(w \cdot x + b)$$

→

2

Medir el error

con la pérdida: MSE o BCE

$$L(y, \hat{y})$$

→

3

Ver la pendiente

el gradiente: hacia dónde baja

$$\partial L \,/\, \partial w$$

→

4

Dar un paso

peso − η · pendiente, y repetir

$$w \leftarrow w - \eta \cdot \partial L / \partial w$$

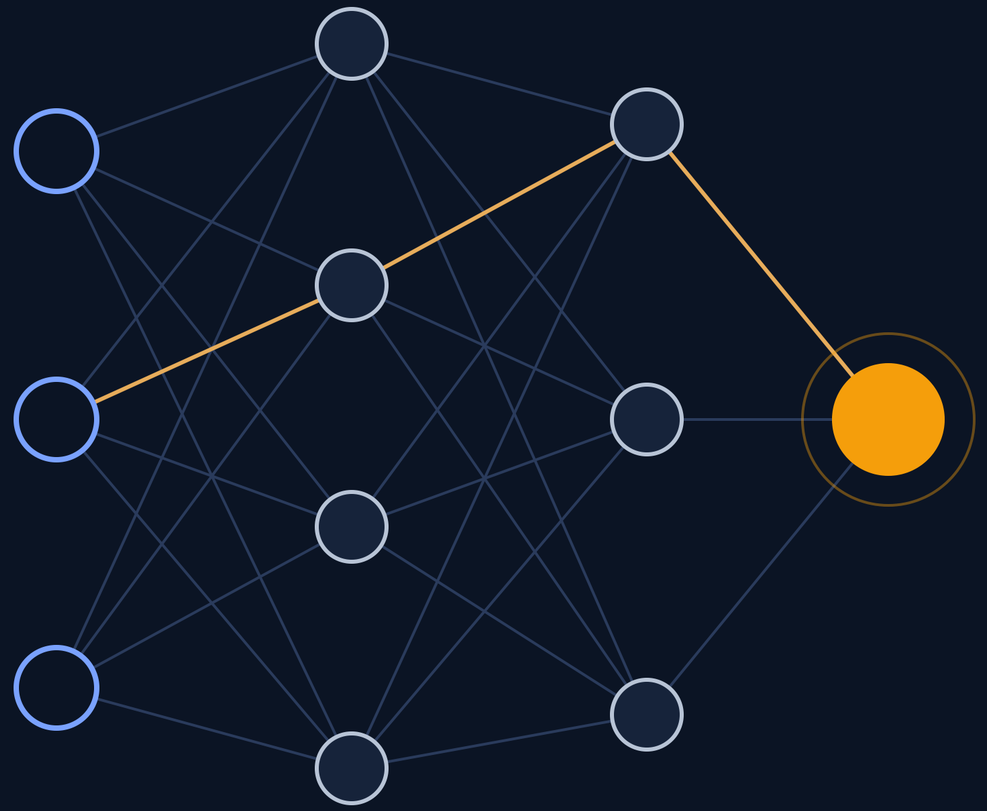
PARA MAÑANA, SÁBADO 08:00

- Terminar el **NB01**: T6–T8 y el Mini-reto 1
- Leer la lectura de tendencia (Canvas)
- Grupos: elegir dataset (cooperativa o propio)

SÁBADO CLASE 1 · SESIÓN 2 · 08:00 – 12:00

De una neurona a una red que aprende

Capas, retropropagación, Keras en tres pasos y el arranque del mini-proyecto



Andrés Ordóñez-Bolaños, MSc

Neural Networks: Deep Learning · UIDE · sábado 24 de octubre de 2026

AGENDA Sábado · 08:00 – 12:00

Cuatro horas, cuatro bloques y un proyecto

08:00	Repaso del viernes	10 min
08:10	05 · De una neurona a una red	20 min
08:30	06 · Retropropagación	40 min
09:10	Pausa	10 min
09:20	07 · Entrenar bien	20 min
09:40	08 · Keras + práctica NB02	35 min
10:15	Pausa	10 min
10:25	09 · El mini-proyecto	15 min
10:40	Trabajo en salas	55 min
11:35	Socialización	12 min
11:47	Quiz final y cierre	13 min



REPASO Sábado · el viernes en un minuto

Lo que ya sabemos

La neurona

Multiplica cada dato por su peso, suma el sesgo y decide con la activación.

Traza una línea

Por eso una sola neurona no puede con XOR, la luz de la escalera.

La pérdida

La nota al revés: MSE para números, BCE para sí o no.

El gradiente

Bajar el Cotopaxi con neblina: peso − η · pendiente.

PREGUNTA PARA EL CHAT

¿Qué problema nos quedó pendiente el viernes, y qué creen que haga falta para resolverlo?

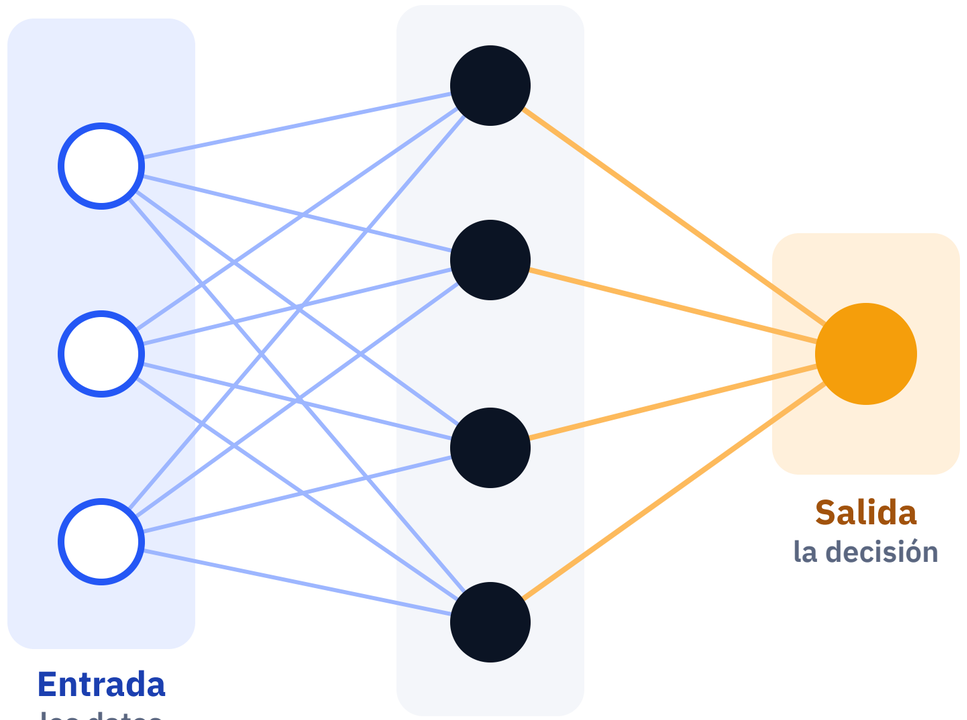
05

De una neurona a una red

Varias neuronas en equipo resuelven lo que una sola no puede

Viernes 01 02 03 04 Sábado **05** 06 07 08 09

Una red = neuronas organizadas en capas



ANALOGÍA: LAS AGENCIAS DE LA COOPERATIVA

Cada **agencia** (neurona oculta) mira la ficha del socio y resume una **pista**:

“¿tiene capacidad de pago?” · “¿es cumplido?” · “¿está sobreendeudado?”

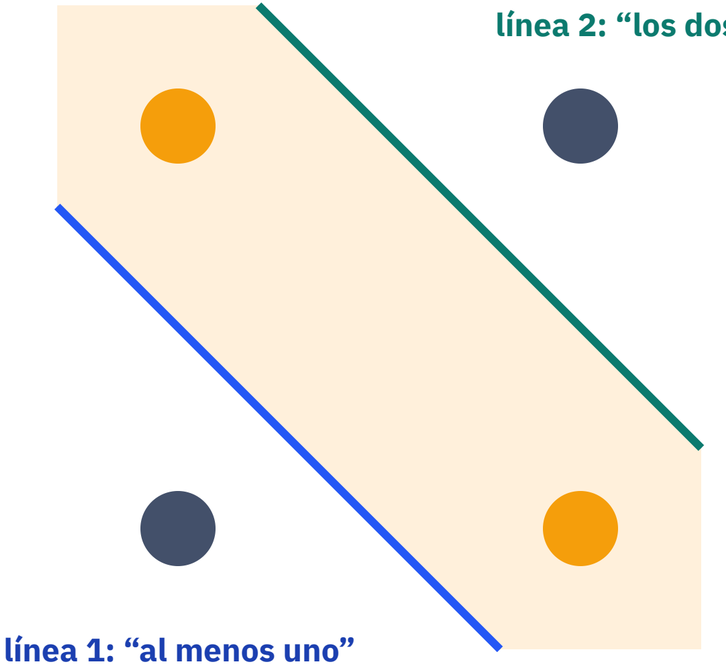
El **comité central** (salida) decide con esas pistas, no con los datos crudos.

EN FÓRMULA, CADA CAPA

$$\mathbf{a} = \varphi(W \cdot \mathbf{x} + \mathbf{b})$$

Cada **fila** de *W* son los pesos de una neurona. **Deep** learning = muchas capas así, una detrás de otra.

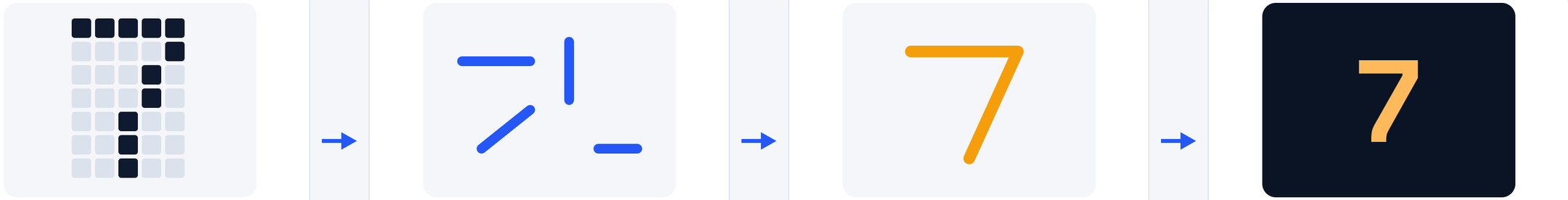
Dos neuronas resuelven la luz de la escalera



interruptores	neurona 1 · ¿al menos uno?	neurona 2 · ¿los dos?	salida · 1 sí y 2 no
0 0	no	no	Apagada
0 1	sí	no	Encendida
1 0	sí	no	Encendida
1 1	sí	sí	Apagada

</> **Abre en Colab:** NB02 · Parte E · Keras aprende XOR: 0, 1, 1, 0

Cada capa arma pistas más elaboradas



EN NUESTRO CASO (UNA TABLA)

ingreso, atrasos, antigüedad… → **pistas**: capacidad de pago, hábito de pago → **riesgo de mora**

PARA CURIOSOS

Con suficientes neuronas, una capa oculta puede aproximar casi cualquier curva (Cybenko, 1989).

¿Cuántos números aprende una red?

LA REGLA

$$\text{entradas} \times \text{neuronas} + \text{neuronas}$$

Cada **conexión** es un peso y cada **neurona** tiene su sesgo.

Ejemplo: 2 entradas y 16 neuronas

$$2 \times 16 + 16 = 48$$

Keras lo cuenta solo con `model.summary()`

Tu turno: ¿cuántos tiene una red 3 → 4 → 1?

RED DE LAS LUNAS · 2 → 16 → 16 → 1

Oculta 1	$2 \times 16 + 16$	**48**
Oculta 2	$16 \times 16 + 16$	**272**
Salida	$16 \times 1 + 1$	**17**
Total		**337**

MINI-PROYECTO · 28 → 32 → 16 → 1

Oculta 1	$28 \times 32 + 32$	**928**
Oculta 2	$32 \times 16 + 16$	**528**
Salida	$16 \times 1 + 1$	**17**
Total		**1 473**

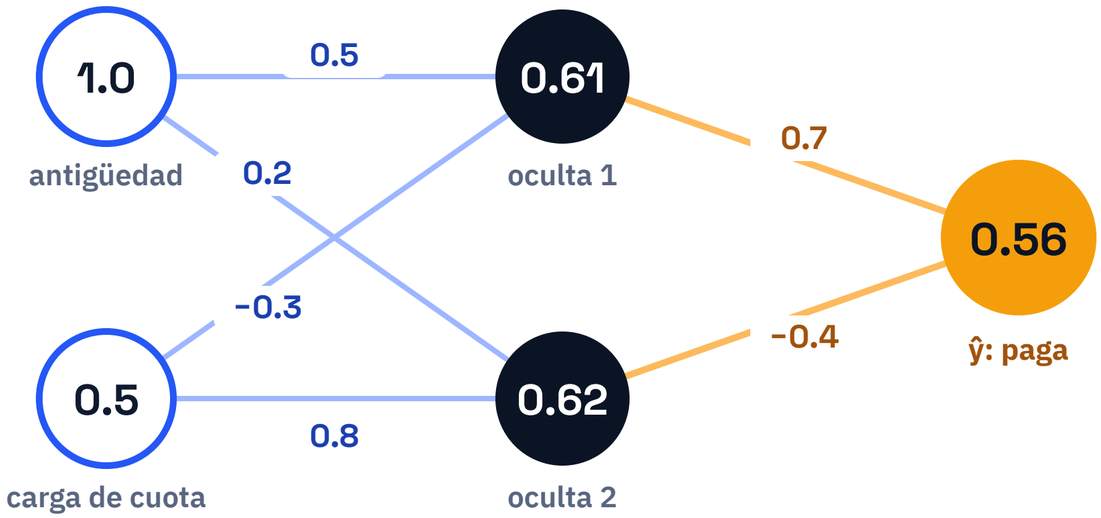
06

Retropropagación

Cómo una red reparte la culpa del error entre todos sus pesos

Viernes 01 02 03 04 Sábado 05 **06** 07 08 09

Paso 1 · Hacia adelante: la red predice



1 **Capa oculta**
suma ponderada y sigmoide: **0.61** y **0.62**

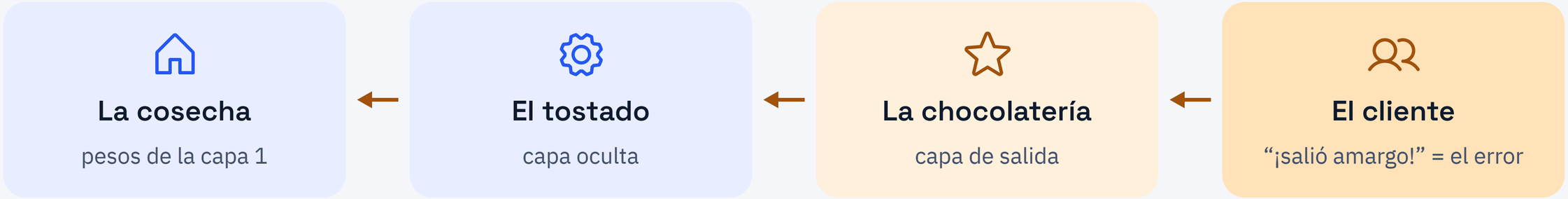
2 **Salida**
con esas dos pistas: **ŷ = 0.56** → “56 % de que paga”

3 **Pérdida**
pero sí pagó (y = 1): –log(0.56) = **0.59**

Tibio: la red debería estar más segura. Hay que corregir.

</> **Abre en Colab:** NB02 · Parte A · Forward

Paso 2 · Hacia atrás: repartir la culpa



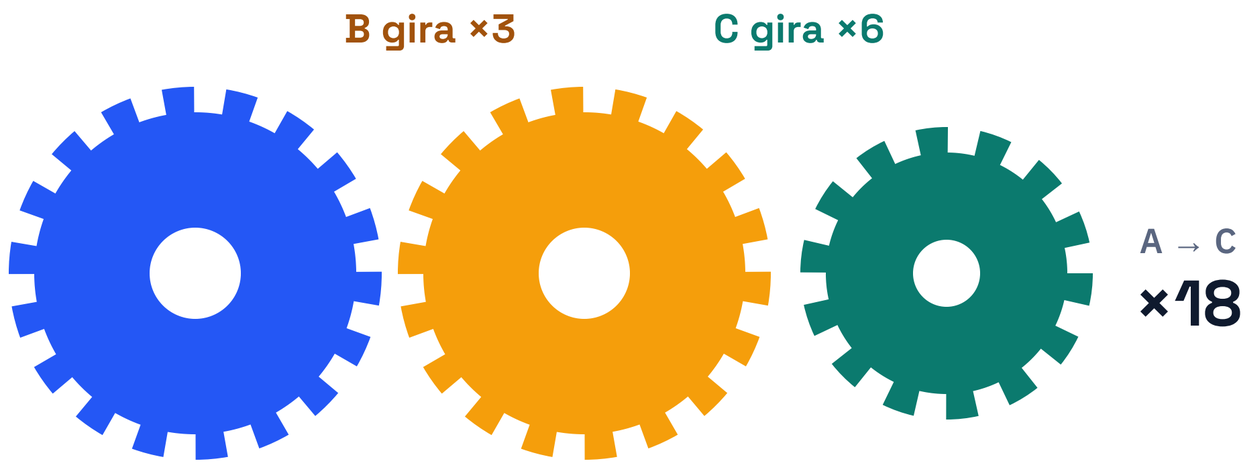
LA IDEA

El error se descubre al final. Cada eslabón recibe culpa **en proporción a cuánto influyó** y pasa su parte hacia atrás.

EN LA RED

culpa de un peso = culpa que llega × cuánto influyó ese peso

La regla de la cadena: engranajes



LA REGLA DE LA CADENA

Si A mueve a B, y B mueve a C, el efecto de A sobre C es la **multiplicación** de los efectos.

PARA CURIOSOS

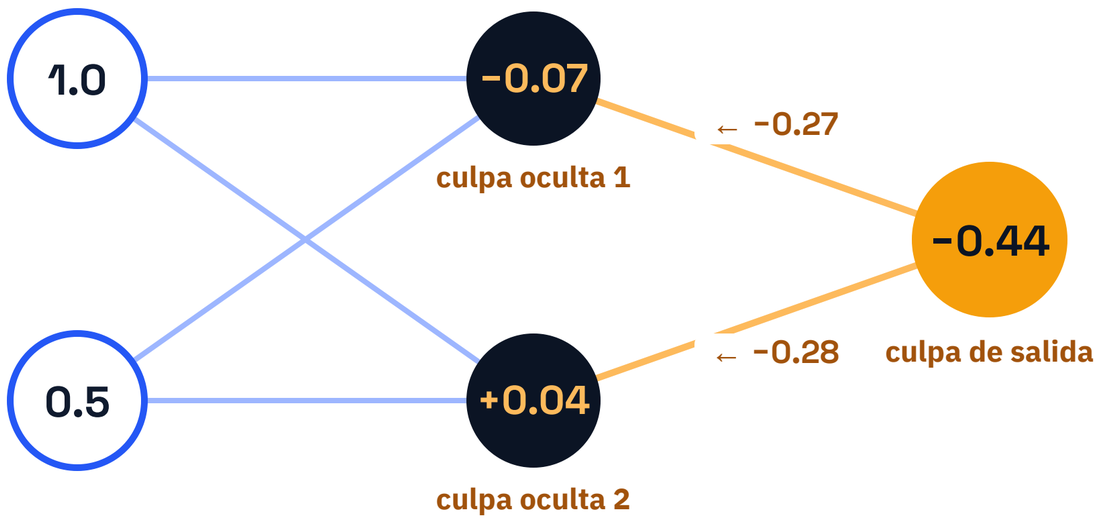
$z = (3x)^2$ en $x = 1$

×3 por dentro, ×6 por fuera → **×18** en total

Retropropagar es multiplicar efectos, eslabón por eslabón, de la salida hacia la entrada.

EJEMPLO 06 · Retropropagación

Hacia atrás, con los números



1 **Culpa de la salida**
ŷ − y = 0.56 − 1 = **−0.44**: le faltó

2 **Culpa de sus pesos**
culpa × lo que entró: −0.44 × 0.61 = **−0.27**

3 **Pasa hacia atrás**
a cada oculta, según su peso y su pendiente: **−0.07** y **+0.04**

Culpa negativa = “súbeme”. Positiva = “bájame”.

</> **Abre en Colab:** NB02 · Parte B · Backward

Paso 3 · Actualizar: un paso cuesta abajo

$$\underbrace{\text{peso}}_{\text{nuevo}} \leftarrow \underbrace{\text{peso}}_{\text{actual}} - \underbrace{\eta}_{\text{largo del paso = 0.5}} \times \underbrace{\text{culpa}}_{\text{la del paso 2}}$$

UN PESO, CON NÚMEROS

$$0.7 - 0.5 \times (-0.27) = \mathbf{0.84}$$

Culpa negativa → el peso **sube**. Igual con los 9 pesos de la red, todos a la vez.

PREDICCIÓN PARA EL SOCIO QUE PAGÓ



↑ más segura de la respuesta correcta

PÉRDIDA



↓ bajó en un solo paso

</> **Abre en Colab:** NB02 · Parte C · un paso y su verificación (gradient check ≈ 10^{-11} ✓)

La retropropagación en cuatro reglas

		EN SÍMBOLOS
1	Culpa de la salida = **predicción – realidad**	$\delta = \hat{y} - y$
2	Culpa de una neurona oculta = culpa que le llega × su peso × su pendiente	$\delta = (W^{T}\delta) \odot \varphi'(z)$
3	Culpa de un peso = culpa de su neurona × lo que entra por ese cable	$\partial L/\partial w = \delta \cdot a$
4	Actualizar: **peso – η × culpa**, y repetir	$w \leftarrow w - \eta \cdot \partial L/\partial w$

PRÁCTICA 06 · Retropropagación

TensorFlow hace la cuenta por ti

ASÍ SE ESCRIBE

```
with tf.GradientTape() as tape:
    y_hat = red(x)                # forward: se graba
    loss  = -tf.math.log(y_hat)
grads = tape.gradient(loss, pesos) # backward automático
```

COMPARADO CON NUESTRA CUENTA A MANO

0.000 000 000 000 000 06

de diferencia: idénticas

ANALOGÍA: UNA GRABADORA

GradientTape **graba** todo lo que pasa hacia adelante y luego lo **reproduce al revés** para calcular las culpas.

Ya no calcularás culpas a mano. Entenderlas sirve para **depurar** cuando algo falla.

Abre en Colab: NB02 · Parte D · TensorFlow hace la cuenta por ti

PAUSA ACTIVA 06 · Verificación rápida

¿Quedó clara la retropropagación?

1. Para predecir la información viaja hacia ___. La culpa viaja hacia ___.

2. Una neurona ReLU dio **z negativo**. ¿Cuánta culpa reciben sus pesos con ese ejemplo?

3. La culpa de un peso es **negativa**. ¿El peso sube o baja en la actualización?

4. ¿Para qué sirve **GradientTape** en TensorFlow?

Respondan en el chat con el número de la pregunta. Tienen 3 minutos.

07

Entrenar bien

Trampas comunes, lotes, optimizadores y una receta para empezar

Viernes 01 02 03 04 Sábado 05 06 **07** 08 09

Tres trampas típicas, y cómo evitarlas

La señal se apaga

Como el teléfono dañado: tras muchas capas, el mensaje llega borroso.

Síntoma: las primeras capas casi no aprenden.

REMEDIO
Usar **ReLU** en las capas ocultas.

Pesos iniciales malos

Si todos empiezan igual, todas las neuronas aprenden lo mismo: clones.

Síntoma: la red no mejora, o explota desde el inicio.

REMEDIO
Inicialización aleatoria y escalada: **he_normal** con ReLU.

Paso η equivocado

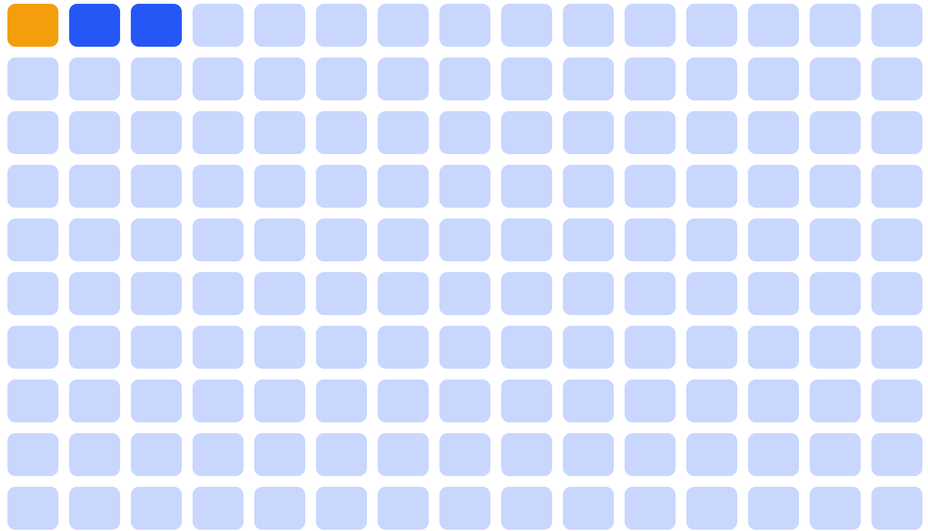
Medio salero en el locro: te pasas una y otra vez.

Síntoma: la pérdida sube, salta o aparece **NaN**.

REMEDIO
Empezar con **Adam** y learning_rate 0.001; si falla, bajarlo.

ANALOGÍA 07 · Entrenar bien

Aprender por lotes: censo o encuesta



1 lote = 32 socios = 1 paso

150 lotes = 4 800 socios = 1 época

ANALOGÍA: CENSO O ENCUESTA

Usar todos los datos en cada paso es un **censo**: exacto pero lento.

Usar un lote es una **encuesta**, como las del INEC: barata y casi igual de buena.

VOCABULARIO DE KERAS

batch_size: cuántos ejemplos por lote (32 es buen inicio)

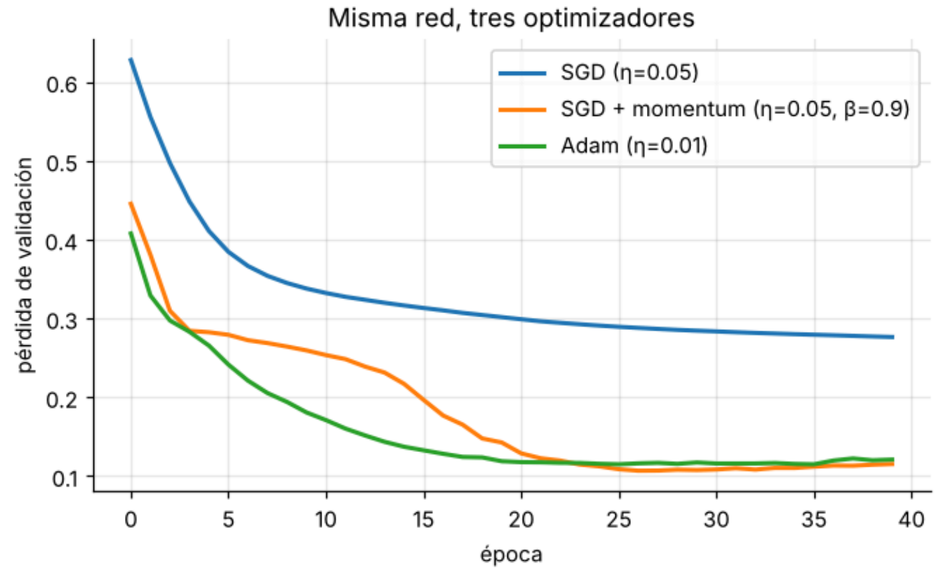
epochs: cuántas vueltas completas a los datos

EN FÓRMULA

gradiente del lote = promedio de las culpas de sus 32 socios

ANALOGÍA 07 · Entrenar bien · gráfica del NB02, Parte G

Tres formas de bajar la montaña



SGD · el caminante

Pasos del mismo largo siempre. Simple, pero lento en valles largos.

Momentum · la bicicleta en bajada

Acumula impulso: acelera en las rectas y no zigzaguea tanto.

Adam · el guía con experiencia

Ajusta el paso de **cada peso** por separado. La opción por defecto.

Lectura de tendencia: ¿hay algo mejor que Adam? Con ajuste justo para todos, las ventajas de los nuevos se achican.

IDEA CLAVE 07 · Entrenar bien

La receta para arrancar (y que funcione)

1 Escala los datos

Que todas las columnas estén en rangos parecidos.

```
StandardScaler()
```

2 Capas ocultas ReLU

Con inicialización He. Empieza con 1 o 2 capas.

```
Dense(32, "relu")
```

3 Salida según la pregunta

¿Sí o no? Una neurona sigmoide y BCE.

```
loss="binary_crossentropy"
```

4 Optimizador Adam

learning_rate 0.001 para empezar.

```
Adam(learning_rate=1e-3)
```

5 Lotes y épocas

batch_size 32. Mira las curvas de entrenamiento y validación.

```
batch_size=32
```

6 Si algo falla

Pérdida que sube o NaN: baja el learning rate.

```
learning_rate ÷ 10
```

08

Keras en tres pasos

Construir, compilar, entrenar: tu primera red en diez líneas

PRÁCTICA 08 · Keras

Keras: construir, compilar, entrenar

1 Construir: apilar capas

```
model = keras.Sequential([
    keras.Input(shape=(2,)),
    keras.layers.Dense(16, activation="relu"),
    keras.layers.Dense(16, activation="relu"),
    keras.layers.Dense(1, activation="sigmoid"),
])
```

Decides la **arquitectura**: cuántas capas, cuántas neuronas y qué activación.

2 Compilar: elegir cómo aprende

```
model.compile(optimizer=keras.optimizers.Adam(learning_rate=0.01),
              loss="binary_crossentropy", metrics=["accuracy"])
```

Decides el **optimizador**, la **pérdida** y qué métrica mirar.

3 Entrenar: repetir el ciclo

```
history = model.fit(X_tr, y_tr, validation_data=(X_va, y_va),
                    epochs=60, batch_size=32)
```

Keras hace adelante → atrás → actualizar, lote a lote y época a época.

PRÁCTICA 08 · Práctica 3 · 25 minutos

Manos a la obra: NB02 completo

		EJEMPLO RESUELTO · DEBERÍAS VER	TU TAREA
A	Forward: la red predice	ŷ = 0.5569	TA · Otro socio
B	Backward: la culpa	δ salida = −0.4431	TB · Culpa de la salida
C	Un paso de aprendizaje	0.5569 → 0.6542	TC · Un paso, socio 2
D	TensorFlow hace la cuenta	diferencia ≈ 0	TD · Engranajes = 18
E	Capa oculta y XOR	0, 1, 1, 0	TE · Cuenta parámetros
F	Keras en 3 pasos	337 parám. · 95 %	Mini-reto 2 · Círculos
G	Tres optimizadores	Adam y Momentum	TG · Lee las curvas

NB02 en Colab

abrir NB02 ↗

Prioridad hoy: A, B, E, F y el **Mini-reto 2**.

09

Mini-proyecto: manos a la obra

Un modelo de riesgo de mora para una cooperativa, de principio a fin

Viernes 01 02 03 04 Sábado 05 06 07 08 **09**

PROYECTO 09 · Mini-proyecto

El reto: anticipar la mora antes de que ocurra

EL CASO

Cooperativa de Ahorro y Crédito “Valle Andino”

Atiende socios en 10 provincias: comerciantes del mercado, agricultores, transportistas, artesanos.

La pregunta: ¿qué socios tienen más riesgo de caer en mora de más de 30 días?

Para qué: acompañarlos **antes** del atraso: una llamada, reestructurar, educación financiera.

8 000
socios

13
variables

12.7 %
cae en mora

LAS VARIABLES

edad · provincia · zona · sector · ingreso · monto · plazo · cuota · antigüedad · créditos previos · atrasos 12 meses · garante · ahorro

Datos 100 % sintéticos. ¿Tu grupo ya tiene los datos de su proyecto de titulación? Úsenlos.

PROYECTO 09 · Mini-proyecto

La ruta del NB03, en nueve paradas



Dado: solo ejecutar y completar pequeños TODO **Lo completa el grupo** **Se redacta**: conclusiones

CHECKPOINTS

Después de cada sección hay un resultado de referencia. Si el suyo se parece, sigan.

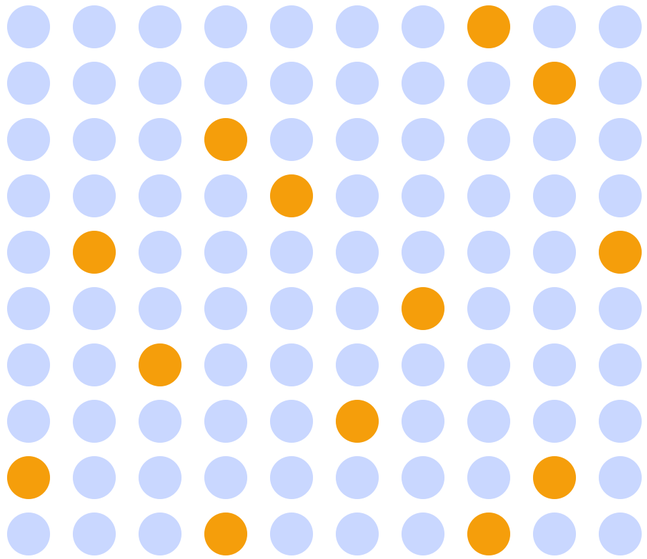
REGLA DE ORO

Los datos de **prueba** se usan **una sola vez**, al final. Todo se decide con validación.

NB03 en Colab

abrir NB03 ↗

Cuidado: 87 % de exactitud puede no valer nada



13 de cada 100 caen en mora

RESULTADOS EN VALIDACIÓN (NB03)

	exactitud	ROC-AUC	detecta
Siempre dice “no mora”	87 %	0.50	0 %
Regresión logística	89 %	0.85	28 %
Perceptrón	89 %	0.84	30 %
Red en Keras	90 %	0.87	45 %

ROC-AUC

Tomo un moroso y un cumplido al azar: ¿con qué probabilidad el modelo le da más riesgo al moroso? **0.5 = moneda al aire**.

DETECTA (RECALL)

De los morosos reales, ¿qué porcentaje encontró el modelo?

PROYECTO 09 · Mini-proyecto

Qué entregan: un notebook y un informe

</> 1 · Notebook .ipynb

- ✓ Corre completo en Colab, sin errores
- ✓ Semilla fija y partición sin trampa
- ✓ Secciones 5 a 8 completas, con salidas
- ✓ Comentarios del código en inglés

2 · Informe PDF · 4 a 6 páginas

- ✓ Problema, datos y métrica elegida
- ✓ Tabla comparando modelos y experimentos
- ✓ Por qué tomaron cada decisión
- ✓ Conexión con su proyecto de titulación

Cierre: martes 27 de octubre (hora exacta en Canvas) · un envío por grupo · **70 puntos**

PROYECTO 09 · Mini-proyecto · rúbrica provisional

Cómo se reparten los 70 puntos



Implementación técnica	25 pts	**Excelente:** Perceptrón, MLP a mano (gradient check aprobado) y Keras; todo corre
Experimentación	15 pts	**Excelente:** Métricas adecuadas a datos desbalanceados; comparación justa
Justificación	15 pts	**Excelente:** Cada decisión argumentada con teoría y datos
Integración al proyecto	10 pts	**Excelente:** Avance claro hacia su proyecto de titulación
Comunicación	5 pts	**Excelente:** Claro, preciso, con figuras legibles

PROYECTO 09 · Mini-proyecto · 55 minutos en salas de Teams

Roles claros, metas por tramo

ROLES · ROTAN CADA SEMANA

Técnico
Comparte el Colab y escribe el código

Datos
Carga, partición y exploración

Experimentos
Lleva la tabla de resultados

Relator
Anota cada decisión y su porqué

METAS POR TRAMO (MINUTOS)

0–10	Datos cargados y partidos (secciones 1–4)
10–25	Perceptrón funcionando (sección 5)
25–42	MLP en Keras entrenando, con curvas (sección 7)
42–50	Primer experimento (sección 8)
50–55	Preparar el resumen de 3 minutos

PROYECTO 09 · Mini-proyecto · 12 minutos · 3 grupos

Tres minutos por grupo, cuatro preguntas

30 s

¿Qué problema y qué datos?

60 s

¿Cuál es su línea base y su mejor resultado?

60 s

Una decisión que discutieron, y por qué la tomaron

30 s

Una duda o bloqueo para resolver entre todos

Los demás grupos: **una pregunta o una sugerencia concreta** por el chat.

QUIZ FINAL Sábado · cierre

Quiz final: ¿cuánto avanzamos?

PARA CERRAR

Quiz final: ¿cuánto avanzamos?

Doce preguntas sobre todo lo de la semana: la neurona, el perceptrón, la pérdida, el gradiente, la retropropagación y Keras.

Al final la página te muestra tu puntaje, **qué repasar** y tu avance frente al pre-quiz del viernes.

Lo que falles, búscalo en la **guía de estudio**: cada pregunta dice a qué tema pertenece.

Escanea y juega

abrir el quiz final ↗

CIERRE Semana 1 · lo que ya saben hacer

De cero a entrenar su primera red

1

Una red son neuronas en capas: juntas trazan fronteras curvas que una sola no puede.

2

Aprender = predecir, medir el error, repartir la culpa hacia atrás y dar un paso.

3

Keras hace el ciclo por nosotros; entenderlo nos permite arreglarlo cuando falla.

ANTES DEL MARTES 27 DE OCTUBRE

- Test teórico · individual
- Encuesta de tendencia · individual
- Entregable del mini-proyecto · grupal

VIERNES 30 · CLASE 2

Arquitecturas: redes densas, convolucionales y recurrentes. ¿Cuál le sirve a sus datos?

RECURSOS Para repasar en casa

Para estudiar: la guía de la semana 1

LA GUÍA DE ESTUDIO

Todo lo de la semana en una página

Los 9 temas, de cero
cada uno con la idea, la fórmula explicada y un ejemplo

Ejercicios con respuesta
resuélvelos y despliega la solución para comprobar

Glosario
las palabras clave del curso en lenguaje sencillo

Enlaces a cada notebook
para volver a la parte exacta en Colab

Abrir la guía

abrir la guía ↗

RECURSOS Para profundizar

Referencias y recursos de la semana 1

BIBLIOGRAFÍA DEL SÍLABO

Nielsen, M. A. (2015). *Neural networks and deep learning*, caps. 1 y 2. Determination Press.

Zhang, A., Lipton, Z. C., Li, M. y Smola, A. J. (2023). *Dive into deep learning*, cap. “Multilayer perceptrons”. Cambridge University Press.

Isola, P., Beery, S. y Bernstein, J. (2023). *6.S898 Deep learning* [curso]. MIT.

ARTÍCULOS MENCIONADOS

Rosenblatt (1958) · Rumelhart, Hinton y Williams (1986) · Cybenko (1989) · He et al. (2015) · Kingma y Ba (2015)

PARA VER Y JUGAR

- playground.tensorflow.org
  una red neuronal en el navegador
- neuralnetworksanddeeplearning.com
  el libro de Nielsen, gratis
- 3blue1brown.com/topics/neural-networks
  videos muy visuales sobre redes y backprop
- d2l.ai
  libro con código ejecutable
- deeplearningweek1.vercel.app/
  la página de la semana: proyectos, guía y quizzes